

CDI Integration

Instances of the repository interfaces are usually created by a container, for which Spring is the most natural choice when working with Spring Data. Spring offers sophisticated support for creating bean instances, as documented in [Creating Repository Instances](#). As of version 1.1.0, Spring Data JPA ships with a custom CDI extension that allows using the repository abstraction in CDI environments. The extension is part of the JAR. To activate it, include the Spring Data JPA JAR on your classpath.

You can now set up the infrastructure by implementing a CDI Producer for the `EntityManagerFactory` and `EntityManager`, as shown in the following example:

```
class EntityManagerFactoryProducer {  
  
    @Produces  
    @ApplicationScoped  
    public EntityManagerFactory createEntityManagerFactory() {  
        return Persistence.createEntityManagerFactory("my-persistence-unit");  
    }  
  
    public void close(@Disposes EntityManagerFactory entityManagerFactory) {  
        entityManagerFactory.close();  
    }  
  
    @Produces  
    @RequestScoped  
    public EntityManager createEntityManager(EntityManagerFactory entityManagerFactory) {  
        return entityManagerFactory.createEntityManager();  
    }  
  
    public void close(@Disposes EntityManager entityManager) {  
        entityManager.close();  
    }  
}
```

JAVA

The necessary setup can vary depending on the JavaEE environment. You may need to do nothing more than redeclare a `EntityManager` as a CDI bean, as follows:

```
class CdiConfig {

    @Produces
    @RequestScoped
    @PersistenceContext
    public EntityManager entityManager;
}
```

In the preceding example, the container has to be capable of creating JPA `EntityManager` itself. All the configuration does is re-export the JPA `EntityManager` as a CDI bean.

The Spring Data JPA CDI extension picks up all available `EntityManager` instances as CDI beans and creates a proxy for a Spring Data repository whenever a bean of a repository type is requested by the container. Thus, obtaining an instance of a Spring Data repository is a matter of declaring an `@Inject` property, as shown in the following example:

```
class RepositoryClient {

    @Inject
    PersonRepository repository;

    public void businessMethod() {
        List<Person> people = repository.findAll();
    }
}
```



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.